

Tema 8.1: Realización de Pruebas en Aplicaciones

Realización de Pruebas

Cuando se está desarrollando un software es esencial realizar las pruebas necesarias para garantizar que el **producto llega al cliente sin errores** y cumpliendo los requisitos solicitados.

En todas las fases de desarrollo de un producto software se pueden cometer errores, algunos de estos errores serán más determinantes que otros y tendrán más o menos implicaciones en el futuro del desarrollo del proyecto.



Realización de Pruebas

Una **prueba** se define como el proceso en el que vamos a ejecutar un software con la intención de descubrir un **error**. Las pruebas deben seguir el razonamiento inverso al utilizado para el desarrollo, de lo contrario, será difícil descubrir los errores cometidos.

Conviene crear pruebas de menor a mayor, es decir, desde una propiedad/método de una clase (**prueba unitaria**) hasta la integración del software (**pruebas de integración**).

El número de pruebas no puede valorarse en función del número de líneas de código, de variables o de valores que se utilicen en el desarrollo del software. Las pruebas serán las representativas de los **casos más importantes o extremos**.



Realización de Pruebas

A nivel general, en el conjunto de un proyecto se distinguen los siguientes tipos de pruebas:

- **Pruebas de comunicaciones.** Comprueba que las interfaces tanto locales como remotas funcionan adecuadamente.
- **Pruebas de rendimiento.** Comprueba los tiempos de respuesta de la aplicación.
- **Pruebas de recuperación.** Se fuerza el fallo del software para comprobar su capacidad de recuperación.
- **Pruebas de volumen.** Se comprueba su funcionamiento con cantidades próximas a su capacidad total de procesamiento

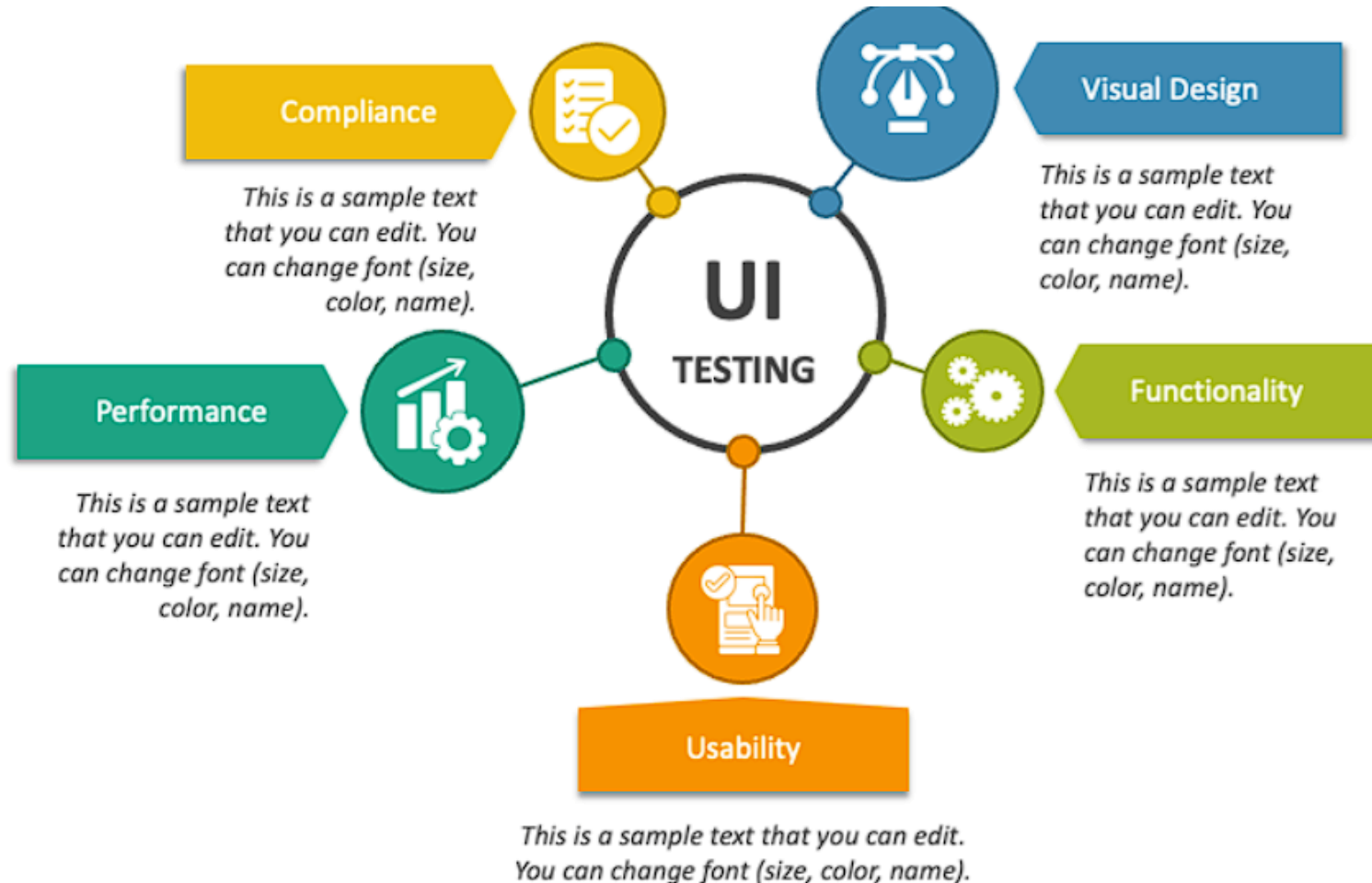
Realización de Pruebas

- **Pruebas de sobrecarga.** Similar al anterior pero en el límite de su capacidad.
- **Pruebas de tensión.** Es similar a la prueba de volumen pero se restringe el tiempo disponible (algo como determinar la velocidad de trabajo).
- **Pruebas de disponibilidad de datos.** Comprueba si tras la recuperación el sistema mantuvo la integridad de los datos.
- **Pruebas de facilidad de uso.** Refiriéndose al usuario final.
- **Pruebas de operación.** Comprueba la correcta implementación de los procedimientos de operación, incluyendo la planificación y control de trabajos, arranque y re arranque del sistema, etc...
- **Pruebas de entorno.** Comprueba la interacción con otros sistemas.

Realización de Pruebas

- **Pruebas de seguridad.** Comprobación de los mecanismos de control de acceso al sistema.
- **Pruebas de usabilidad.** Comprueba “lo utilizable” que es nuestro sistema.
- **Pruebas de almacenamiento.** Comprobación de la cantidad de memoria principal y secundaria que el programa usa y el tamaño de los archivos temporales.
- **Pruebas de configuración.** Aunque a veces resulta imposible se debe comprobar el programa con cada tipo de hardware, sistema operativo, antivirus....
- **Pruebas de instalación.** En especial la automatización para facilidad del usuario final.
- **Pruebas de la documentación.** Hace referencia tanto a la documentación técnica para desarrolladores futuros como a la documentación para el usuario.

Realización de Pruebas



Realización de Pruebas

Recomendaciones:

- ✓ Los casos de prueba deben incluir tanto **entradas correctas como incorrectas** para evaluar el comportamiento del sistema en cualquier situación.
- ✓ Las pruebas deben diseñarse de forma que tengan la **máxima probabilidad** de encontrar el mayor número de errores con la **mínima cantidad de esfuerzo** y tiempo.
- ✓ El **programador** no debería probar sus propios programas.
- ✓ Las pruebas deben centrarse e insistir más en las partes o módulos que **más se utilizan o sean más críticos** para el sistema.
- ✓ No debe asociarse el error a la negligencia de un programador; la finalidad de las pruebas debe ser encontrar errores y no desprestigiar a nadie.

Clasificaciones de los tipos de pruebas

Dentro de todos los tipos de pruebas disponible, podemos separarlas en función del momento en el que se llevan a cabo:

- ☐ Pruebas unitarias.
- ☐ Pruebas de integración.
- ☐ Pruebas de carga.
- ☐ Pruebas de aceptación.

O las relacionadas con la **implementación (puesta en marcha/funcionamiento)** del software:

- ☐ Pruebas unitarias.
- ☐ Pruebas de caja negra.
- ☐ Pruebas de caja blanca o transparente.
- ☐ Revisiones.

Clasificaciones de los tipos de pruebas

Finalmente, una vez la **aplicación ya está desarrollada y validada**, se realizarán las **pruebas de sistema**, que validarán la integración del software **en el sistema ya existente del usuario final o cliente**. En este sentido encontramos pruebas de:

- Rendimiento.
- Resistencia.
- Robustez.
- Seguridad.
- Usabilidad.
- Instalación.



System Testing

Pruebas unitarias

La prueba unitaria se plantea a pequeña escala, y consiste en **ir probando uno por uno (unidad por unidad)** los **diferentes módulos que constituyen una aplicación**.

Existen **herramientas** de software libre que permite automatizar las pruebas:

NUnit: permite la realización de pruebas unitarias para Microsoft .NET y Mono.

Junit: conjunto de clases que permite realizar la ejecución de clases Java, para evaluar si cada uno de los métodos de la clase se comporta como se espera.

Las pruebas unitarias deben ser automáticas, completas, reutilizables o independientes.



Pruebas de caja negra

Comprueban el **funcionamiento de un componente** de software por medio de su interfaz, sin entrar a analizar su funcionamiento interno:

- Se introducen datos correctos.
- Se introducen datos erróneos.
- Se introducen datos equivalentes.
- Se introducen valores extremos superiores e inferiores.

Por ejemplo, se tiene una función que, dados dos números enteros, devuelve cuál es el mayor. Para este tipo de pruebas se debe comprobar la respuesta al introducir:

- Dos números enteros.
- Dos números enteros iguales.
- Dos números reales.
- Uno entero y otro real.
- Un número positivo y uno negativo.
- Un carácter.

Pruebas de caja negra

En todos los casos, la función debe dar una respuesta, aunque sea un **mensaje de error**. Lo que no debe pasar es que la función quede bloqueada o se interrumpa, etc.

Técnicas para establecer juegos de pruebas de caja negra:

- **Clases de equivalencia:** se determinan los diferentes tipos de entrada y salida, agrupándolos y seleccionando los casos de prueba para cada tipo o conjunto de datos de entrada y salida.
- **Análisis de los valores límite:** estudian los valores iniciales y finales, ya que estadísticamente se ha demostrado que tienen más tendencia a detectar errores.
- **Estudio de errores típicos.**
- **Manejo de interfaz gráfica:** deben diseñarse casos de prueba que permitan descubrir errores en el manejo de ventanas, botones, iconos, etc.
- **Datos aleatorios:** se trata de utilizar una herramienta que automatice las pruebas y que genere de forma aleatoria los casos de prueba. Podría utilizarse como complementaria a las anteriores o en casos en que no sea posible aplicar ninguna otra.

Pruebas de caja blanca

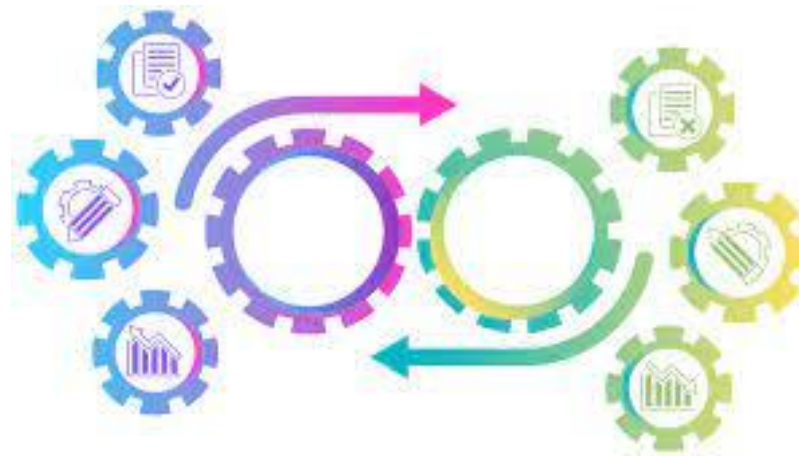
Analizan la forma en que un **componente de software** resuelve un determinado problema atendiendo a los **detalles internos de implementación**. Es decir, examinan el interior de un módulo de software desarrollado:

- **Sentencias, condiciones, bucles...**
- **Estructuración del código.**
- **Optimización del código.**
- Los juegos de pruebas deberían prever todos los casos posibles, todos los posibles caminos del **flujo de control del software implementado**.
- Un método para poder establecer de forma adecuada los juegos de pruebas es el método de cobertura de flujo de control de una estructura de control.
- Haciendo el camino inverso de lo que se suele hacer a la hora de programar se puede buscar la obtención de un diagrama de flujo de control que represente el código y **probar todos los caminos simples, todas las condiciones y todos los bucles del programa**.

Pruebas de integración

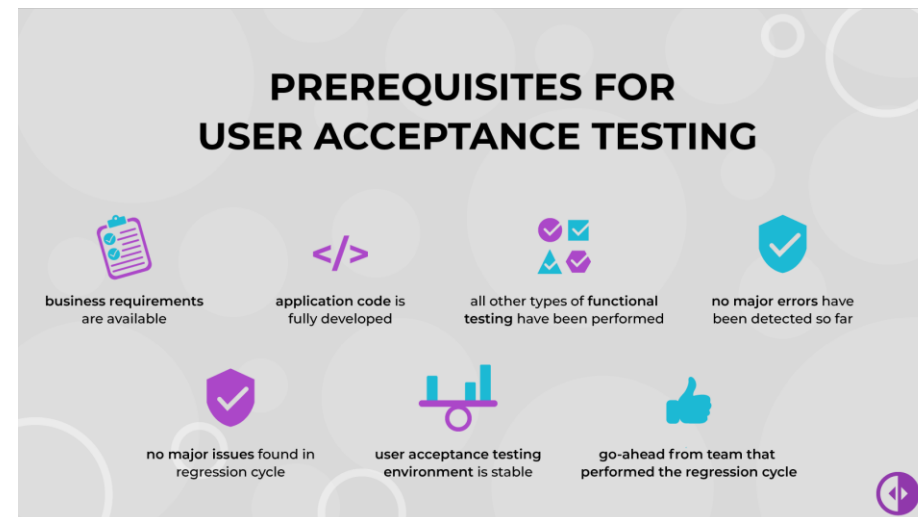
Verifican que un gran **número de partes del software (módulos de la aplicación desarrollada) siguen funcionando cuando éstos se han juntado**. A veces se produce la situación de que unos componentes que funcionaban perfectamente por separado, al integrarse, fallan.

Este tipo de pruebas se suele realizar de forma gradual. Se prueban dos componentes juntos. Si funcionan, se incorpora un tercero, y así de forma sucesiva.

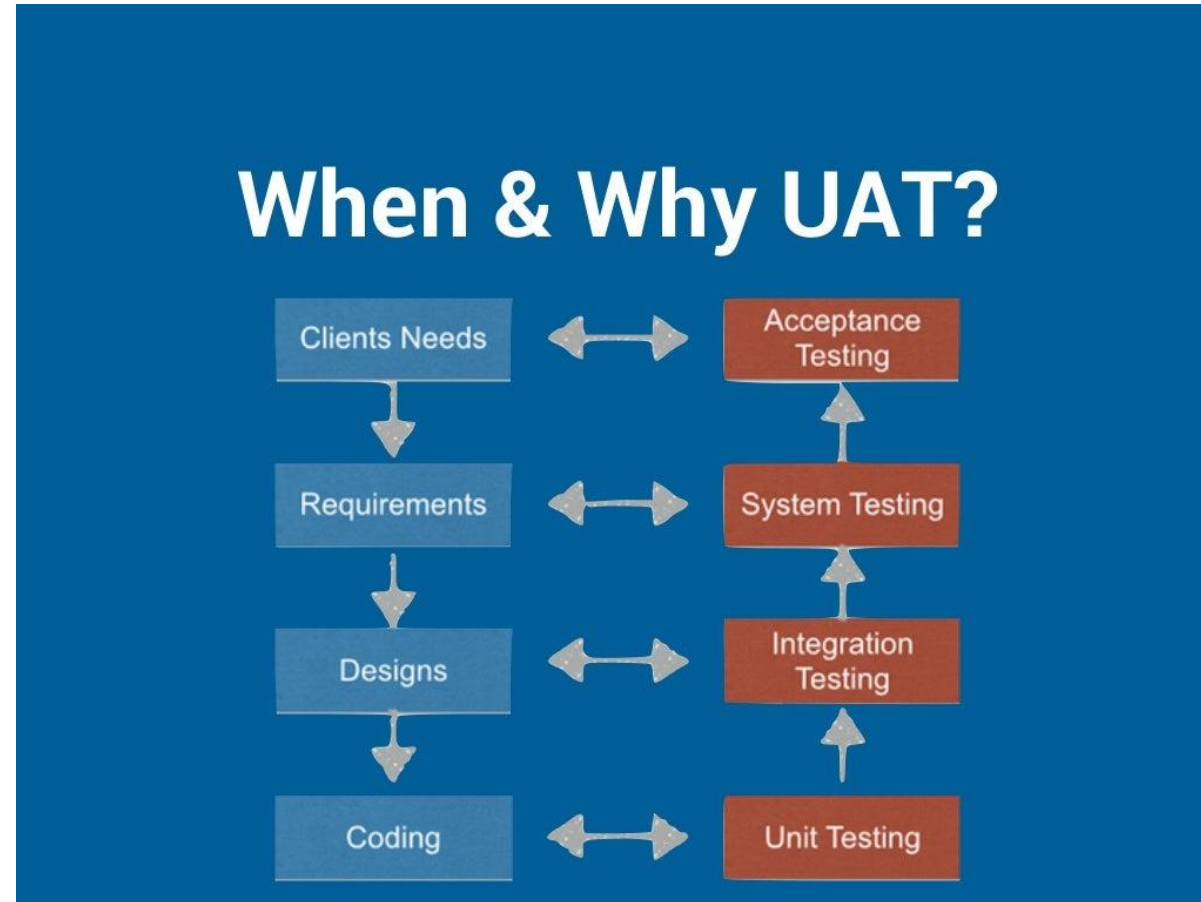


Pruebas de aceptación

- El objetivo de las pruebas de aceptación es **validar que un sistema cumple con el funcionamiento esperado** y permitir al usuario de dicho sistema que determine su aceptación, desde el punto de vista de su funcionalidad y rendimiento.
- Las pruebas de aceptación son definidas por el usuario del sistema y preparadas por el equipo de desarrollo, aunque **la ejecución y aprobación final corresponden al usuario**

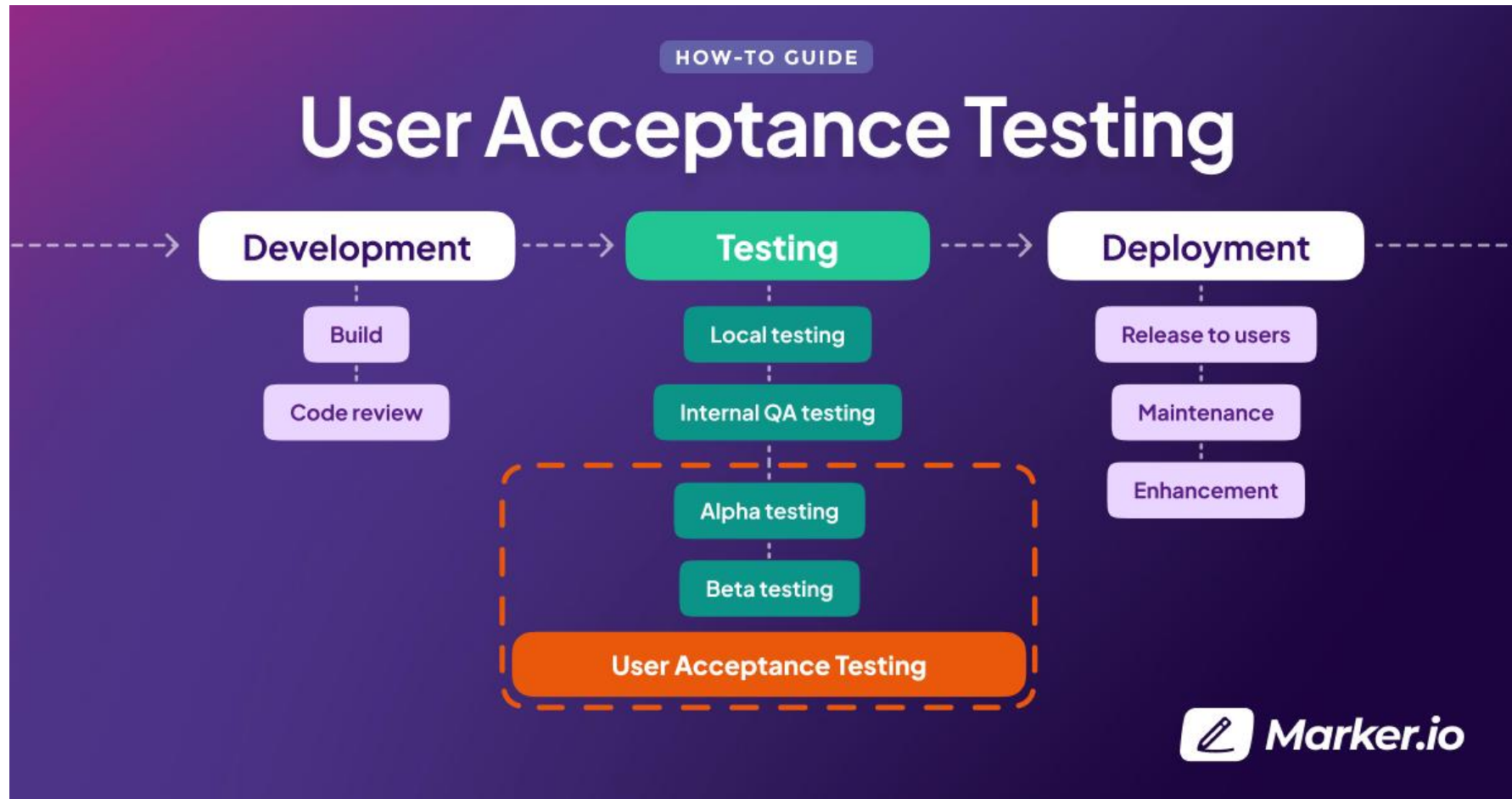


Pruebas de aceptación



VERSIONES ALFA Y BETA

- **Prueba alfa: (versión alfa)** se lleva a cabo, por un cliente, en el lugar de desarrollo. Se usa el software de forma natural con el desarrollador como observador del usuario y registrando los errores y problemas de uso. Las pruebas alfa se llevan a cabo en un entorno controlado.
- **Prueba beta: (versión beta)** se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. A diferencia de la prueba alfa, el desarrollador no está presente normalmente. Así, la prueba beta es una aplicación en vivo del software en un entorno que no puede ser controlado por el desarrollador. El cliente registra todos los problemas que encuentra durante la prueba beta e informa a intervalos regulares al desarrollador.



Os dejo algunos enlaces de interés:

<https://docs.microsoft.com/es-es/visualstudio/test/getting-started-with-unit-testing?view=vs-2022&tabs=dotnet%2Cmstest>

<https://docs.microsoft.com/es-es/visualstudio/test/unit-test-basics?view=vs-2022#createunit-test-projects-and-test-methods-c>
<https://www.manning.com/books/unit-testing>